
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 2

Introdução à Programação em C

Exercícios de Autorrevisão (2.1–2.6)

Resolvidos passo a passo, abordagem incremental
Capítulos 1 + 2

Construindo sobre o Capítulo 1

Bem-vindo ao Capítulo 2 – o primeiro capítulo onde *escrevemos código*! Construindo sobre os conceitos do Cap. 1 (hardware, software, ambientes de desenvolvimento), agora introduzimos:

- **Estrutura básica** de um programa em C: `#include`, `main`, chaves, instruções, retorno
- **Funções da biblioteca-padrão** `<stdio.h>`: `printf`, `scanf`
- **Variáveis e tipos** (`int`, `por enquanto`)
- **Operadores aritméticos**: `+`, `-`, `*`, `/`, `%`
- **Tomada de decisão** com `if` e operadores relacionais/de igualdade
- **Sequências de escape**: `\n`, `\t`, etc.
- **Especificadores de conversão**: `%d` para inteiros

Boa prática de programação

Este é o capítulo mais importante para iniciantes. Cada conceito aqui será usado em *todos* os capítulos seguintes. Não se apresse – experimente, faça pequenas variações, observe os erros do compilador. Cada erro é uma oportunidade de aprendizado.

Contents

1	Exercício 2.1 – Preenchimento de Lacunas	2
2	Exercício 2.2 – Verdadeiro ou Falso	5
3	Exercício 2.3 – Escrever Instruções em C	8
4	Exercício 2.4 – Cálculo do Produto de 3 Inteiros	9
5	Exercício 2.5 – Programa Completo	9
6	Exercício 2.6 – Identificar e Corrigir Erros	11

1. Exercício 2.1 – Preenchimento de Lacunas

Item (a)

Resposta detalhada

Resposta: Todo programa em C inicia sua execução pela função `main`. A função `main` é o *ponto de entrada* de todo programa em C. Não importa quantas outras funções você defina, a execução sempre começa em `main`.

```
1  int main( void )
2  {
3      /* primeira instrucao executada esta aqui */
4      return 0;
5  }
```

Item (b)

Resposta detalhada

Resposta: A **chave à esquerda** (`{`) inicia o corpo de cada função, e a **chave à direita** (`}`) encerra o corpo.

As chaves `{ }` delimitam blocos de código em C. Esse padrão é amplamente usado em outras linguagens derivadas: C++, Java, JavaScript, C#.

Item (c)

Resposta detalhada

Resposta: Toda instrução termina com um(a) **ponto e vírgula** (`;`).

Erro comum de programação

Esquecer o `;` no final de uma instrução é o erro mais comum em C iniciantes. O compilador geralmente reporta o erro na *próxima* linha (não na linha do erro real), o que pode confundir.

Item (d)

Resposta detalhada

Resposta: A função `printf` da biblioteca-padrão exibe informações na tela. `printf` (de *print formatted*) é a função mais usada em C para exibir texto. Está declarada em `<stdio.h>`.

Item (e)

Resposta detalhada

Resposta: A sequência `\n` representa o caractere de **nova linha** (*newline*), que posiciona o cursor no início da próxima linha.

Sequências de escape são caracteres especiais precedidos por `\`:

- `\n` – nova linha
- `\t` – tab horizontal
- `\\` – backslash literal
- `\"` – aspa dupla literal
- `\a` – alerta (beep)

Item (f)

Resposta detalhada

Resposta: A função `scanf` é usada para obter dados do teclado. `scanf` (de *scan formatted*) lê dados formatados da entrada-padrão (`stdin`). Exige o operador de endereço `&` antes da variável destino (com algumas exceções a serem vistas nos Caps. 6 e 7).

```
1 int idade;  
2 scanf( "%d", &idade ); /* note o & */
```

Item (g)

Resposta detalhada

Resposta: O especificador `%d` é usado para inteiros, tanto em `scanf` quanto em `printf`.

Outros especificadores que veremos:

- `%d` – inteiro decimal com sinal
- `%f` – ponto flutuante (Cap. 3)
- `%c` – caractere (Cap. 8)
- `%s` – string (Cap. 8)

Item (h)

Resposta detalhada

Resposta: Quando um novo valor sobrescreve um existente, dizemos que o processo é **destrutivo**.

```
1 int x = 10;  
2 x = 20; /* destrutivo: o 10 foi perdido para sempre */
```

Item (i)

Resposta detalhada

Resposta: Quando lemos um valor da memória, ele permanece intacto – esse processo é **não destrutivo**.

```
1  int x = 10;  
2  int y;  
3  y = x;  /* le x (nao destrutivo): x continua 10 */
```

Item (j)

Resposta detalhada

Resposta: A instrução `if` é usada para tomada de decisões.

```
1  if ( idade >= 18 ) {  
2      printf( "Maior de idade.\n" );  
3  }
```

A condição usa operadores relacionais (<, >, <=, >=) ou de igualdade (==, !=).

2. Exercício 2.2 – Verdadeiro ou Falso

Item (a) – `printf` sempre começa em nova linha

FALSO

`printf` começa onde o *cursor* estiver. Se a impressão anterior não terminou com `\n`, a nova saída continua na mesma linha.

```
1 printf( "Ola, " );    /* imprime "Ola, " sem \n */
2 printf( "Mundo!\n" ); /* imprime "Mundo!" na MESMA
    linha */
```

Saída: Ola, Mundo!

Item (b) – Comentários geram saída

FALSO

Comentários **não produzem nenhuma ação** em tempo de execução. Eles são *ignorados pelo compilador* e servem apenas para documentar o código para leitores humanos.

```
1 /* Este comentario NAO aparece na tela */
2 // Este tambem nao
```

Item (c) – `\n` posiciona cursor na próxima linha

VERDADEIRO

A sequência `\n` faz exatamente isso.

Item (d) – Variáveis precisam ser declaradas antes do uso

VERDADEIRO

C é *fortemente tipado*: toda variável precisa de declaração explícita antes de ser usada (com nome e tipo).

Item (e) – Todas variáveis precisam de tipo

VERDADEIRO

Cada variável em C tem um tipo (`int`, `double`, `char`, ...) que determina quanta memória ocupa e como é interpretada.

Item (f) – número e NúMeRo são iguais

FALSO

C é **case-sensitive**: `número`, `NúMeRo`, `NUMERO` e `numero` são quatro variáveis distintas para o compilador.

Boa prática de programação

Use uma convenção consistente. Em C, a tradição é `camelCase` para variáveis e funções, `MAIUSCULAS` para constantes (`#define`).

Item (g) – Declarações em qualquer ponto da função

FALSO (em C89)

No padrão **C89** (clássico), declarações devem aparecer no início do bloco, antes de qualquer instrução executável.

A partir do **C99** essa restrição foi removida – declarações podem aparecer em qualquer lugar, como em C++. Mas o livro segue C89 estrito, então marca como FALSO.

Item (h) – printf precisa de &

FALSO

É o oposto: `scanf` normalmente exige `&` antes de variáveis; `printf` **não**.

```
1 int x = 42;
2 printf( "%d\n", x );      /* SEM & */
3 scanf(  "%d", &x );      /* COM & */
```

Intuição: `printf` só precisa do *valor* (para imprimir); `scanf` precisa do *endereço* (para escrever de volta).

Item (i) – % só com inteiros

VERDADEIRO

O operador `%` (módulo, resto da divisão) exige operandos **inteiros**. Para ponto flutuante, há `fmod` em `<math.h>`.

Item (j) – *, /, %, +, - mesma precedência

FALSO

Há dois níveis de precedência:

- **Maior:** *, /, % (multiplicação, divisão, módulo)
- **Menor:** +, - (adição, subtração)

Como em álgebra: $2 + 3 * 4 = 14$, não 20.

Item (k) – Identificadores muito longos idênticos

FALSO

O padrão C garante apenas 31 caracteres significativos em identificadores internos. Compiladores mais antigos podem tratar os nomes *distintos* acima como o *mesmo* nome (apenas os primeiros 31 chars contam). Compiladores modernos geralmente diferenciam, mas a portabilidade não é garantida.

Boa prática de programação

Use nomes *significativos* mas *razoavelmente curtos*: `contador`, `saldoMedio`, `numeroAlunos`. Identificadores com mais de 31 caracteres são tipicamente um sinal de que o nome deveria ser refeito.

Item (l) – 3 linhas exigem 3 printf

FALSO

Um único `printf` com `\n` pode produzir várias linhas:

```
1 printf( "Linha 1\nLinha 2\nLinha 3\n" );
```

Saída:

Saída do programa

```
Linha 1
Linha 2
Linha 3
```


3. Exercício 2.3 – Escrever Instruções em C

Resposta detalhada

a) Declarar 4 variáveis int:

```
1 int c, estaVariavel, q76354, numero;
```

b) Pedir um inteiro:

```
1 printf( "Digite um inteiro: " );
```

Note o espaço após : e a ausência de `\n` (o cursor fica na mesma linha aguardando entrada).

c) Ler inteiro para a:

```
1 scanf( "%d", &a );
```

d) Se numero != 7, exibir mensagem:

```
1 if ( numero != 7 ) {  
2     printf( "A variavel numero nao e igual a 7.\n"  
3     );  
3 }
```

e) Imprimir mensagem em uma linha:

```
1 printf( "Este e um programa em C.\n" );
```

f) Em duas linhas, quebrando após “C”:

```
1 printf( "Este e um programa em\nC.\n" );
```

g) Cada palavra em uma linha:

```
1 printf( "Este\ne\num\nprograma\nem\nC.\n" );
```

h) Palavras separadas por tabulações:

```
1 printf( "Este\te\tum\tprograma\tem\tC.\n" );
```

Boa prática de programação

`\n` e `\t` são suas ferramentas principais para formatar saída no início. Brinque com elas: combine, alterne, observe como o cursor se move.

4. Exercício 2.4 – Cálculo do Produto de 3 Inteiros

Resposta detalhada

a) Comentário documentando propósito:

```
1  /* Calcula o produto de tres inteiros */
```

b) Declarar variáveis:

```
1  int x, y, z, resultado;
```

c) Pedir 3 inteiros:

```
1  printf( "Digite tres inteiros: " );
```

d) Ler 3 inteiros:

```
1  scanf( "%d%d%d", &x, &y, &z );
```

e) Calcular produto:

```
1  resultado = x * y * z;
```

f) Exibir resultado:

```
1  printf( "O produto e %d\n", resultado );
```

5. Exercício 2.5 – Programa Completo

Resposta detalhada

```
1  /* Ex2.5: produto_tres.c
2     Calcula o produto de tres inteiros */
3  #include <stdio.h>
4
5  int main( void )
6  {
7     int x, y, z, resultado; /* declara variaveis */
8
9     printf( "Digite tres inteiros: " ); /* prompt */
10    scanf( "%d%d%d", &x, &y, &z );      /* le tres
11                                     inteiros */
12    resultado = x * y * z;              /*
13                                     multiplica */
14    printf( "O produto e %d\n", resultado ); /* exibe
15                                     */
16
17    return 0;
18 } /* fim da funcao main */
```

Exemplo de execução:

Saída do programa

```
Digite tres inteiros: 2 3 4  
O produto e 24
```

Nota: `scanf("%d%d%d", ...)` aceita os 3 números separados por espaços, tabs ou novas linhas – todos servem como delimitadores.

6. Exercício 2.6 – Identificar e Corrigir Erros

Item (a)

Resposta detalhada

Código com erro:

```
1 printf( "0 valor e %d\n", &numero );
```

Erro: Uso do operador de endereço & em printf. printf precisa do *valor*, não do endereço.

Correção:

```
1 printf( "0 valor e %d\n", numero );
```

(O endereço só seria necessário com o especificador %p para imprimir o endereço em si, conceito do Cap. 7.)

Item (b)

Resposta detalhada

Código com erro:

```
1 scanf( "%d%d", &numero1, numero2 );
```

Erro: Falta o & antes de numero2. scanf precisa do endereço para gravar de volta.

Correção:

```
1 scanf( "%d%d", &numero1, &numero2 );
```

Erro comum de programação

Esquecer o & no scanf é um dos erros mais comuns. O programa pode até compilar (geralmente com warning), mas vai gravar em endereço aleatório, causando comportamento imprevisível ou falha de segmentação.

Item (c)

Resposta detalhada

Código com erro:

```
1 if ( c < 7 );{  
2     printf( "C e menor que 7\n" );  
3 }
```

Erro: Ponto e vírgula após a condição do if. Isso cria uma *instrução vazia* como corpo do if, e o bloco com chaves sempre executa,

independentemente da condição!

Correção:

```
1  if ( c < 7 ) { /* sem ponto e virgula */
2      printf( "C e menor que 7\n" );
3  }
```

Erro comum de programação

Esse é um bug **silencioso** – o programa compila e “funciona”, mas com comportamento errado. Sempre revise os if para garantir que não haja ; acidental após a condição.

Item (d)

Resposta detalhada

Código com erro:

```
1  if ( c => 7 ) {
2      printf( "C e igual ou menor que 7\n" );
3  }
```

Erros:

- => não é operador válido. Em C, “maior ou igual” é >=.
- A mensagem está semanticamente errada: “maior ou igual”, não “igual ou menor”.

Correção:

```
1  if ( c >= 7 ) {
2      printf( "C e maior ou igual a 7\n" );
3  }
```

Operadores relacionais corretos em C:

Operador	Significado
<	menor que
>	maior que
<=	menor ou igual
>=	maior ou igual
==	igual
!=	diferente

Erro comum de programação

Confundir = (atribuição) com == (comparação de igualdade) é outro erro frequente. `if (x = 5)` sempre é verdadeiro (atribui 5 a x e avalia a 5, que é diferente de zero). O correto seria `if (x == 5)`.

Gabarito Rápido – Capítulo 2: Autorrevisão

Respostas Resumidas

2.1:

- a) `main` b) `{ e }` c) `;` d) `printf`
e) nova linha f) `scanf` g) `%d` h) destrutivo i) não destrutivo j) `if`

2.2 – Verdadeiro/Falso:

- a) F (imprime onde o cursor está)
b) F (comentários são ignorados em execução)
c) V d) V e) V
f) F (C é case-sensitive)
g) F em C89 h) F (é o contrário)
i) V j) F (dois níveis de precedência)
k) F (só 31 chars garantidos) l) F (`\n` permite várias linhas)

2.3: `int c, ...; printf("Digite ..."); scanf("%d", &a); if (numero != 7) {...};` combinações de `\n` e `\t` para formatar.

2.4: comentário inicial, declaração, `prompt`, `scanf` com 3 `&`, cálculo, `printf` com resultado.

2.5: programa completo de 13 linhas com `#include`, `main`, declaração, `prompt`, leitura, cálculo, saída, `return`.

2.6 – 4 erros corrigidos:

- a) Remover `&` em `printf`
b) Adicionar `&` em `numero2`
c) Remover `;` após condição do `if`
d) `=>` $\rightarrow$ `>=`